

**Universidad Nacional Mar del Plata - Facultad de
Ingeniería Departamento de Informática - Tecnologías
Informáticas B**

Consignas de Práctica: Suite de Pruebas API

En base a los tests de ejemplo y leyendo previamente la tutoría N° 6 que explica la sintaxis para crear sus propios tests resolver:

Parte I Corrección de los tests de ejemplo

Deberá corregir los tests de las líneas 19 y 34 que están escritos en el script: [authTests.js](#)

De modo tal que no cambie su comportamiento o prueba realizada, sino que evalúe la respuesta del servidor de manera correcta para que el color del botón resulte en verde al presionarlo.

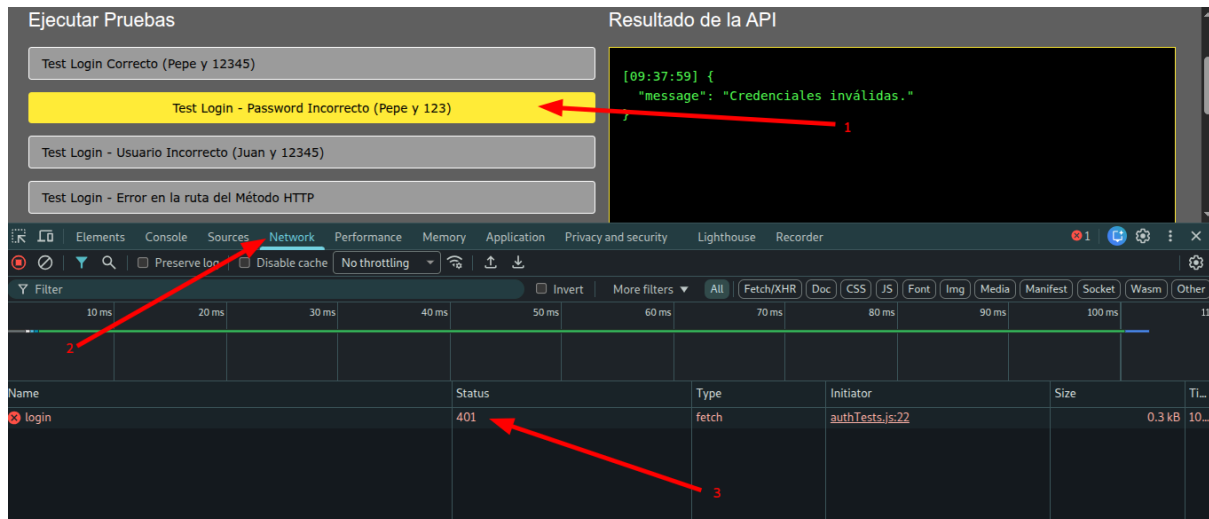
Vamos a explicar con código el caso que prueba el Password Incorrecto. Se debe modificar solamente la sección de código de [authTests.js](#) donde se evalúa la respuesta del servidor. Primero revisemos el test que evalúa el uso de un Password incorrecto para el usuario Pepe:

```
testUtils.createTestButton("Test Login - Password Incorrecto (Pepe y 123)",
  async (btn) => {
    const response = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username: 'pepe', password: '123' })
    });

    const data = await response.json();
    testUtils.log(data);

    if (response.ok) {
      testUtils.setSuccess(btn);
    }
  });
```

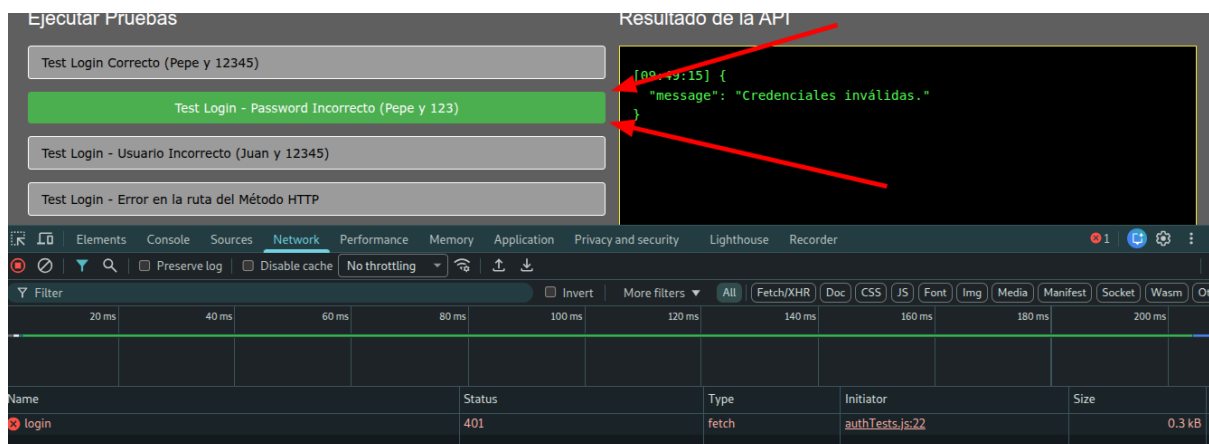
Si verificamos el código de respuesta del servidor al pulsar el botón del test (1). En el inspector del navegador en la pestaña network (2) y en la columna Status (3) veremos el resultado 401. El mismo corresponde a HTTP 401 Unauthorized (No Autorizado)



Entonces solo hará falta corregir la sentencia condicional del if a `response.status === 401` para que el botón resulte en verde al ejecutar el test:

```
if (response.status === 401) {
  testUtils.setSuccess(btn);
}
```

Resultado en el navegador:



Parte II Creación de tests adicionales

1. Pruebas de Autenticación: Registro de Usuarios

Objetivo: Practicar el envío de datos mediante el método `POST` imitando la lógica del login.

- **Tarea:** Crear un test llamado **"Test Registro - Usuario Nuevo"**.
- **Instrucciones:** Debe realizar una petición `POST` a `/api/auth/register` enviando un JSON con un `username` y `password` nuevos.
- **Validación:** Si el servidor responde con un código `201` (Creado), marcar el botón como exitoso usando `testUtils.setSuccess(btn)`.

"Pista: Para que el test sea reutilizable, intenta que el username sea dinámico (por ejemplo, sumándole la fecha actual) para evitar errores de duplicados"

2. Pruebas de Seguridad: El Intruso (403 Forbidden)

Objetivo: Verificar que un usuario con rol "producer" no puede acceder a rutas de administrador.

- **Tarea:** Crear un test llamado **"Test Seguridad - Productor accediendo a Admin"**.
- **Instrucciones:**
 1. Primero, realizar un login con las credenciales de 'pepe'.
 2. Usar el token obtenido para intentar hacer un `GET` a la ruta `/api/admin/users`.
- **Validación:** El test se considera exitoso si la consola muestra un error de "Acceso denegado" y el status de la respuesta es `403`.

Sugerencia: La consigna dice "realizar un login con las credenciales de 'pepe'". Como ahora creaste la función `okLogin()`, podrías unificar el lenguaje y decir: "Asegurar la sesión llamando a la función `okLogin()` (la cual usa las credenciales de 'pepe')".

3. Pruebas de Recursos: Eliminación Dinámica de Samples

Objetivo: Practicar el flujo de datos entre diferentes endpoints (Data Flow), el uso del verbo HTTP `DELETE` y el manejo de parámetros dinámicos en la URL.

Tarea: Crear un test llamado "**Test Eliminar Sample Dinámico**".

Instrucciones:

1. **Asegurar Sesión:** Iniciar el test llamando a la función `okLogin()` para garantizar que existe un token válido en el `localStorage`.
2. **Obtener Identificador:** Antes de borrar, realizar una petición `GET` a `/api/samples/my-samples` para obtener la lista actual de archivos del usuario.
3. **Validar Existencia:**
 - o Si la lista está vacía, mostrar un mensaje de advertencia en la consola indicando que se debe subir un sample primero.
 - o Si hay samples, extraer el `id` del primer elemento de la lista.
4. **Ejecutar Eliminación:** Realizar una petición con el método `DELETE` a la ruta `/api/samples/${targetId}`, inyectando el ID obtenido en el paso anterior.

Validación:

- El test se considera exitoso si la API responde con un código de estado `200` u `ok`, confirmando que el registro y el archivo físico fueron removidos.
- Utilizar `testUtils.log()` para mostrar en pantalla qué ID específico se intentó borrar.

4. Pruebas de Validación: Carga Incompleta

Objetivo: Probar la robustez del servidor ante datos faltantes.

- **Tarea:** Crear un test llamado "**Test Subir Sample - Error por Datos Faltantes**".
- **Instrucciones:**
 1. Utilizar `FormData` para simular la subida de un archivo.
 2. Agregar el archivo de audio con `blob`, pero **no** agregar el campo `category` o `display_name`.
- **Validación:** El servidor debería rechazar la petición con un error `400`. El test es correcto si el botón detecta ese error de validación.

Tips para los estudiantes:

- Siempre que sea necesario iniciar el test llamando a la función `okLogin()` para garantizar que existe un token válido en el `localStorage`.
- No olviden usar siempre `testUtils.log(data)` para visualizar la respuesta JSON en la consola de la página.
- Recuerden que para las rutas protegidas, es obligatorio incluir el encabezado `Authorization: Bearer <token>`.
- Observen cómo el estado del botón cambia a amarillo (cargando) y luego a verde (éxito) o rojo (error) según el resultado.